

The Control Server:

Robot Entry Point & Dispatch Integration for the WashU GDG Delivery Fleet

Jaxon Poentis

WashU GDG Delivery Robot Project — fleet-platform

August 11, 2026

Status: proposed Companion to docs/DESIGN.md (D1–D12)

Abstract

This proposal specifies the *control server*: the single, always-on server every delivery robot connects to on boot, and the spine through which the club’s order-matching, path-planning, and teleoperation systems interact with the physical fleet. The control server is the `server/` component of the open-source *fleet-platform*; the club’s delivery system (`delivery-gdg-platform`) runs on top of it through SDKs. We describe the connection lifecycle, the availability and dispatch flows, upstream telemetry, the teleoperation and intervention model, the graph-learning feedback loop, and the bridge between the platform’s event bus and the club’s Kafka-based services.

1 Summary

The control server is the single server every robot dials out to on boot: connection fabric, presence, telemetry fan-out, message bus, intervention queue, WebRTC signaling, and replay — one process, one box. For the club deployment it carries the full delivery loop:

1. Robot boots → outbound WSS to `fleet.wudelivery.<tld>` → authenticated with its per-robot credential, capability manifest registered, presence *online*.
2. Club services (matcher, path service, command brain) learn the robot is available through SDK subscriptions to platform events.
3. A match and waypoint path, computed club-side, is delivered to the robot as an *assignment* payload over the bus; the robot’s club node parses it and executes it leg by leg with Nav2. Execution is fully abstracted from the platform.
4. The robot streams pose, state, and battery telemetry up the same socket; the platform fans it out to the operations console and to subscribed services.
5. On autonomy failure the robot raises help → intervention queue → an operator claims an exclusive *lease* → peer-to-peer WebRTC teleop (video + velocity setpoints) → handback → the brain replans.
6. Per-leg traversal reports flow as domain events from the robot to the path service, which updates the campus graph’s edge weights.
7. A small *bridge service* forwards platform events to the club’s Kafka topics for services that consume them there.

2 Role in the System

The control server owns exactly the platform vocabulary: connections, manifests, presence, telemetry, channels, the intervention queue, leases, signaling, replay. It is the source of truth for *who is connected*, *who has authority over which robot*, and *what each robot can do*.

Everything delivery-shaped — orders, matching, pathing, the campus graph — lives in `delivery-gdg-platform` as services that connect to the platform like any other client. The club’s decision-making role is the *command brain*: a club-side service built on the TypeScript SDK that accepts no inbound connections — purely a client, holding a single outbound WebSocket to the control server. It digests presence, telemetry, and events into a world model, decides (match, route, reroute), and publishes — `send(robot, channel, payload)` per-robot or `broadcast(channel, payload)` fleet-wide.

This replaces the club’s current `apps/command` and the socket-hub portion of `apps/authoritative`; the matcher, orders, and the web application stay club-side unchanged.

2.1 Why the separation is worth the extra hop

Splitting brain from control server means an assignment travels brain → control server → robot rather than brain → robot. The extra hop is a deliberate trade: cheap where it is paid, valuable in what it buys.

What the hop costs. The added leg is brain → control server: two server-side processes on the same VPS or in the same region, so roughly a millisecond plus an in-memory socket lookup. The slow leg of any message to a robot is the radio link (campus Wi-Fi or LTE, 15–100 ms), and that leg exists in every possible design. The traffic riding this path — assignments, presence, telemetry, edge reports — is latency-tolerant: whether a waypoint list arrives in 40 ms or 45 ms changes nothing. The one flow where latency genuinely matters, teleop video and twist, bypasses the server entirely (peer-to-peer WebRTC; the server only performs signaling). The hop is paid exactly where it is free.

What the hop buys. Robots are behind NAT and must dial out to *something*; the real alternatives are worse:

- *Brain as the socket server* — the brain would own WSS accept, auth, heartbeats, reconnects, presence, and backpressure alongside delivery logic. That is the current `apps/authoritative` architecture, hub bugs included, and every delivery-logic change would redeploy the process robots depend on for connectivity and safety.
- *Robots hold two connections* (control server for teleop, brain for assignments) — two presence truths that can disagree, duplicated reconnect logic, and a flaky robot link spent twice.

With the split there is one connection fabric and one source of truth for presence, leases, and authority; the brain is a decision process that can crash, redeploy, or be rewritten mid-semester without dropping a single robot connection (and can rebuild its world model on restart — §4); and the control server stays generic enough to open-source. This is the trade made by every authoritative game server, by Slack’s message servers, and by Tailscale’s DERP relays: clients connect to dumb, stable fabric; smart services sit behind it.

It also keeps horizontal scaling open. Because the brain addresses robots by identity (`send("robot_42", ...)`), never by socket, the connection fabric can later become *multiple* control servers behind a routing layer — look up which server holds the robot, forward the bytes — with

zero changes to the brain, the SDK, the robot agent, or the protocol. The stateful core (presence, queue, leases) is what makes naive replication unsafe: exactly-one-driver requires atomic state transitions, so two servers must never both reason about the same robot’s lease. The designed path (D11, tier 3) is therefore *shard by fleet* — each fleet’s robots, queue, and leases live whole on one server, keeping every authority decision local and atomic while capacity scales across fleets. Version 1 deliberately runs a single instance: one small VPS carries hundreds of robots (~1 KB/s per robot; teleop is peer-to-peer, so a session costs the server about as much as a chat message), and the components that hit limits first — TURN relay bandwidth, spectator fan-out — are dumb ones (coturn, later an SFU) that already multiply independently.

Next steps on this axis: none in version 1 beyond keeping the seam clean — the **store/** interface (sqlite → postgres) is drawn now and built never, and nothing in **ops/** may assume it can see more than one fleet’s state “for convenience.” Fleet-sharding gets built only when a real deployment saturates a single instance; at that point it is packaging and routing work, not a protocol or client change.

3 Connection Lifecycle

3.1 Reaching the server

`fleet.wudelivery.<tld>` resolves (A/CNAME) to the box running **fleet-server** and **coturn** — the tier-1 Docker Compose deployment (D11). TLS terminates at the server with ordinary WebPKI certificates. The main **wudelivery** site can live anywhere else; the fleet subdomain is its own record.

3.2 Identity and enrollment

Every robot holds a *per-robot credential*; identity is always derived server-side from the credential, never claimed by the client. Provisioning is a two-step exchange, the Tailscale auth-key pattern:

1. An admin creates a *fleet enrollment key* in the console (scoped, expiring, revocable).
2. On first boot the robot presents the enrollment key; the server registers it, mints an opaque per-robot token, and returns it. The robot stores the token and uses it for every subsequent connection.

Tokens are opaque random strings stored hashed in the platform database (the robots/tokens runtime state of D1) — revocable the instant a robot is lost, stolen, or compromised, which stateless formats such as JWT handle badly. A compromised robot therefore compromises exactly one identity, not the fleet. (Signed stateless credentials become worth revisiting at tier 3, when verification may happen on more than one server.)

3.3 Handshake

robot	control server
-- WSS connect ----->	
-- hello {token, agent version} -->	auth: token lookup -> robot_id + fleet
<-- welcome {robot_id, time} -----	identity confirmed, never claimed
-- manifest {drive, streams, ...}->	registry: capabilities -> console UI
<-- ack -----	presence: ONLINE (event on the bus)
<==== heartbeats (both ways) =====>	silence => presence lost

The manifest declares the drive contract (`{drive: {type: "twist", max_v, max_w}}`), camera streams, battery, and the *domain channels* the robot's club node speaks (e.g. `assignment`, `edge_report`). The console renders only what the manifest declares.

Presence is heartbeat-based with expiry, not socket-lifecycle — and this applies to *every* client class: robots, services, and operator consoles alike. Reconnection is the robot agent's job: exponential backoff, same token. A lease outlives a network blip shorter than its TTL; the robot-side deadman covers the gap.

4 Availability and Matching

On connect and disconnect the platform emits typed presence events (`robot.online`, `robot.offline`) on its bus, and every intervention transition is likewise a typed lifecycle event (`robot.help_requested`, `robot.lease_granted`, `robot.lease_released`). Every SDK subscription opens with a *state snapshot* — current presence, state machine state, and lease holders for the fleet — followed by the live event stream. Snapshot-then-stream is what makes the brain genuinely restartable: a freshly deployed brain reconstructs its world model from the snapshot plus its own club-side state (orders in flight), rather than depending on having witnessed every event since the beginning of time.

From platform presence, live telemetry, and club state (orders in flight, battery, current state machine state) the brain derives which robots are *dispatchable* and feeds that to the matcher.

The world model branches off from the club's state manager (`apps/authoritative/internal/state/`) — the robot + order registry with the delivery lifecycle (`IDLE → ASSIGNED → MOVING_TO_PICKUP → AT_PICKUP → MOVING_TO_DROPOFF → AT_DROPOFF → RETURNING`), whose `GetAvailableRobots` query is exactly this dispatchability derivation. Today that code is a state *store* — raw setters, no transition rules; inside the brain it becomes a state *machine*: enforced legal transitions, plus the dispatch loop around it (order created + robot available → matcher → path service → `send(robot, "assignment", ...)`). Its inputs swap from the old gRPC/WS hub to SDK subscriptions, and its own `IsOnline` tracking retires in favor of platform presence — one liveness truth. The two vocabularies stay disjoint: the delivery statuses are the club's state machine, the platform's is `AUTONOMOUS | HELP_REQUESTED | TELEOP`; a robot can be `MOVING_TO_DROPOFF` and `AUTONOMOUS` at once, and neither leaks into the other.

This is the integration seam: club services consume platform *events*, not platform internals. The platform's database describes the platform's world (fleets, tokens, robots, leases); the club's database describes the club's world (users, orders, deliveries).

5 Dispatch: Delivering the Match and Path

When the matcher pairs an order with a robot and the path service produces a waypoint list (server-side A* over the campus graph with live travel-time weights):

```
command brain --- send(robot_42, "assignment",
                        {order, waypoints[], deadline}) ---> platform bus
platform bus --- routes bytes; size/rate limits -----> robot's socket
club node    --- parses assignment; sequences Nav2 goals leg by leg
```

- The assignment payload rides an opaque domain channel; its schema is club property, versioned in `delivery-gdg-platform`. Nothing order- or path-shaped enters `protocol/`.

- The bus is at-most-once by design: that keeps the platform dumb, and reliable delivery is the domain’s contract to provide where it needs it. Here, the brain re-sends until the club node confirms on the same channel, and assignments are idempotent club-side. The TypeScript SDK ships a small request/ack helper so each service does not reinvent this pattern.
- On the robot: generic `fleet_agent` (platform SDK) $\leftrightarrow$ thin club node (~50 lines: assignment parsing, leg sequencing, escalation policy) $\leftrightarrow$ Nav2 and the base. Navigation and SLAM are entirely the robot stack’s concern.

6 Telemetry Upstream

Over the same socket, continuously:

- **Typed platform telemetry:** frame-relative pose (geographic frame on campus, local Cartesian indoors), velocity, battery, state machine state (`AUTONOMOUS` | `HELP_REQUESTED` | `TELEOP` | ...), health flags (including teleop media link health during a session). Consumed by the console map, fleet list, replay recorder, and the brain’s world model.
- **Typed help:** `request_help {reason, context}` $\rightarrow$ intervention queue (§7).
- **Domain events on channels:** `edge_report {edge_id, seconds, blocked?}` emitted by the club node as each leg completes. The path service subscribes and updates edge weights, so the campus graph learns from fleet history.

Per-robot steady load is ~ 1 KB/s — chat-user class. Bus-enforced size and rate limits keep the control plane light; video never rides this pipe.

7 Teleoperation and Intervention

1. The robot’s club node (escalation policy is club code) decides it needs help and sends `request_help`. Server state machine: `AUTONOMOUS` $\rightarrow$ `HELP_REQUESTED`; a queue entry appears on every operator console. Day-one metric: help raised $\rightarrow$ first operator eyeball.
2. An operator claims the robot; the server issues an exclusive, expiring, renewable *lease*; state $\rightarrow$ `TELEOP`. Steal = revoke + reissue, never share. Every state transition is a server-side action; the peer-to-peer channel never carries authority.
3. The server performs WebRTC signaling only. Video and velocity setpoints flow peer-to-peer (operator browser $\leftrightarrow$ robot, both dialing out; ICE races LAN, hole-punched, and TURN paths). Target: under 200 ms glass-to-glass.
4. **Robot-side arbitration is layered.** The `fleet_agent` accepts twist commands only when they bear the current lease identifier, and publishes them to the highest-priority input of the robot’s twist mux — so even a buggy club node cannot fight the operator at the actuator. On entering `TELEOP`, the club node additionally cancels its active Nav2 goal: autonomy must not keep planning against a robot that is not obeying it, and — the real hazard — a stale goal left alive would reclaim the mux the instant the operator releases, lurching the robot toward wherever it was headed before the takeover. Autonomy re-arms only when a fresh goal is issued after handback (step 6). Throughout, local obstacle avoidance stays active (safeguarded velocity control) and a ~ 300 ms deadman runs: no valid twist $\rightarrow$ zero velocity. Physics never waits on the network.

5. **Operator loss is the server’s transition, not the robot’s.** Operator consoles are heartbeated presence clients like everything else; if the operator’s presence lapses or the lease goes unrenewed, the server revokes the lease and returns the robot to `HELP_REQUESTED` — back in the queue for the next operator. The robot’s role during the gap is to fail safe (the deadman has already stopped it) and to report media-link loss as telemetry; it never re-queues itself off peer-to-peer state. The asymmetry matters because the WebRTC path can die while the operator’s WSS is healthy (a TURN hiccup): if the robot decided for itself, one robot could hold a live lease *and* sit in the queue — exactly the two-drivers ambiguity leases exist to kill.
6. **Handback always replans** — no “did the operator go too far?” threshold. On resolve, the server transitions the robot to `AUTONOMOUS`; the club node reports its leg invalidated along with current pose; the brain has the path service route fresh from the nearest graph node and dispatches a new assignment. If the operator barely moved the robot, the replan degenerates to “continue to the next waypoint” — same code path, no special cases. This also keeps the operator out of graph surgery: no console UI for picking waypoints, which would be domain knowledge in the platform. Every intervention is black-box recorded for scrubbable replay.

Operators can also open teleop *proactively* on any robot (claim from `AUTONOMOUS`, same lease mechanics, same arbitration in step 4) — for example when the brain or an anomaly highlight in the console (pose unchanged for 60s while nominally autonomous) suggests a robot is silently stuck. The *judgment* that a delivery is off-track (no progress along its assigned leg, blown ETA) is the brain’s: it watches pose against assignments and either directs its club node to raise help or flags an operator.

8 Cross-Service Communication: The Kafka Bridge

Club services keep speaking Kafka to each other (`apps/authoritative` already has the plumbing). A ~30-line club-side *bridge service*, built on the TypeScript SDK, is the single point where the two messaging worlds touch — one direction in version 1:

```
platform bus --- TS SDK bridge (~30 lines) ---> club Kafka
  robot.online / pose / edge_report --> topics for robot_manager et al.
```

The reverse path (Kafka topics re-published as bus sends) is deliberately deferred: a durable, replayable log feeding an at-most-once command bus means a bridge restart can replay stale commands at robots. If a club service needs to command a robot, it uses the SDK directly. Rule of thumb: club-to-club traffic rides Kafka; anything robot-facing or operator-visible rides the platform.

9 End-to-End Sequence

```
robot boots -> DNS -> WSS -> hello/auth -> manifest -> ONLINE
                                     |
               command brain (SDK) <-- robot.online +
user orders -> club web/API -> matcher pairs order + robot
path service -> A* over campus graph -> waypoint list
brain -> send(robot, "assignment", {...}) -> bus -> club node -> Nav2 legs
robot -> pose/state telemetry -> console map, replay, brain world model
robot -> edge_report per leg -> path service -> graph weights update
```

```
... a leg fails ...  
robot -> request_help -> queue -> operator claims -> lease -> P2P teleop  
operator drives it clear -> handback -> AUTONOMOUS -> leg invalidated  
brain -> replan from nearest node -> fresh assignment -> Nav2 legs resume  
delivery completes -> club-side order close-out
```

The platform half of this sequence is roadmap steps 1–2 against the sim. The club half (matcher beyond FIFO, path service, brain) is club-side work built during the semester on top of it — the D5 audit is the honest baseline for how much of it exists today.

10 Build Order

1. **Protocol v0** (protocol/): envelope + hello (per-robot token) + enrollment exchange, manifest, twist + lease, frame-relative pose/telemetry, help/lease lifecycle messages, snapshot-then-stream subscribe semantics, layer declaration. Domain channels come free with the envelope.
2. **Vertical slice** (roadmap 1–2): server skeleton, sim robot, console — connect, see pose, claim, WASD drive, handback — with zero hardware, before the club semester starts (~2026-08-21).
3. **Club integration** (roadmap 4): bridge service and brain migration; matcher and path service consume presence via SDK/bridge; `apps/command` retired.
4. **Real hardware** (roadmap 3): encoder feedback, `/cmd_vel` path, twist mux, and Nav2 bring-up on the robot, then `fleet_agent` plus the thin club node.